

Analyse d'un réseau de trajets avec Spark (PySpark)

Low-Level LakeHouse with Spark — Projet individuel

Ahmed Abidhiah — MSc Data Engineering & Cloud Computing, 2e année

2025-2026

Code : <https://github.com/abidhiah/spark-reseau-trajets>

1. Introduction et environnement

Une entreprise de mobilité veut exploiter trois mois de trajets (janvier à mars 2026) entre 36 stations réparties sur 3 zones. L'objectif est de construire une chaîne de traitement reproductible avec Spark : lecture et contrôle qualité des CSV bruts, calcul d'indicateurs avec l'API RDD puis l'API DataFrame, stockage analytique en Parquet, analyse des plans d'exécution et construction d'un graphe orienté du réseau. Le projet est réalisé en **PySpark**, dans un seul langage de bout en bout.

Élément	Version
PySpark (pip) / moteur Spark affiché	3.5.3 / 3.5.0
Python	3.12
Java	17.0.12
Mode d'exécution	local[*] (macOS)

Commande de lancement : voir README.

2. Ingestion et qualité des données

2.1 Structure des enregistrements

Chaque ligne valide devient un `namedtuple` Python `Trajet` (dans `src/commun.py`) avec des champs nommés et typés après conversion :

Champ	Type après conversion
<code>trajet_id, station_depart, station_arrivee, type_abonnement</code>	<code>str</code>
<code>date_heure</code>	<code>datetime</code> (format <code>%Y-%m-%d %H:%M:%S</code>)
<code>duree_min, distance_km</code>	<code>float</code>

Les stations sont lues dans un `namedtuple` `Station` (zone en `int`, coordonnées en `float`). La fonction `valider()` convertit une ligne et renvoie soit ("`OK`", `Trajet`), soit ("`REJET`", `motif`), ce qui permet de séparer les lignes valides et rejetées avec un simple `filter`. La liste des identifiants de stations est diffusée aux workers avec `sc.broadcast` pour le contrôle des stations connues.

2.2 Règles de rejet

Les règles sont appliquées dans l'ordre ci-dessous ; une ligne reçoit le motif de la première règle qu'elle viole, ce qui garantit `valides + rejetées = lignes lues`.

Ordre	Motif	Règle
1	<code>format_invalide</code>	7 champs non vides attendus
2	<code>date_invalide</code>	<code>date_heure</code> au format <code>yyyy-MM-dd HH:mm:ss</code>
3	<code>hors_pperiode</code>	date comprise entre le 01/01/2026 et le 31/03/2026 (les trois mois étudiés)
4	<code>station_inconnue</code>	départ et arrivée présents dans <code>stations.csv</code>
5	<code>valeur_non_numerique</code>	durée et distance convertibles en nombre fini (<code>nan</code> et <code>inf</code> refusés)
6	<code>valeur_non_positive</code>	durée > 0 et distance > 0
7	<code>boucle</code>	station de départ ≠ station d'arrivée

8	abonnement_inconnu	type_abonnement ∈ {annuel, mensuel, occasionnel}
9	doublon_id	trajet_id unique (voir règle ci-dessous)

Point de vigilance : en Python, `float("nan")` et `float("inf")` ne lèvent pas d'erreur, et `nan <= 0` vaut `False`. Sans le test `math.isfinite`, une durée `nan` passerait toutes les règles. Les règles 3 et 8 ne rejettent aucune ligne dans ce fichier, mais elles protègent le pipeline contre des données futures.

Tests unitaires. `tests/test_regles.py` vérifie 15 cas limites sans Spark : un cas par motif, `nan`, `inf`, distance négative, bornes exactes de la période, et l'ordre de priorité des règles. Résultat : 15/15.

```

OK ligne correcte          attendu=None obtenu=None
OK 6 champs               attendu=format_invalide obtenu=format_invalide
OK champ vide             attendu=format_invalide obtenu=format_invalide
OK date illisible         attendu=date_invalide obtenu=date_invalide
OK date avant la période  attendu=hors_période obtenu=hors_période
OK date après la période  attendu=hors_période obtenu=hors_période
OK station inconnue       attendu=station_inconnue obtenu=station_inconnue
OK durée texte            attendu=valeur_non_numerique obtenu=valeur_non_numerique
OK durée NaN              attendu=valeur_non_numerique obtenu=valeur_non_numerique
OK distance infinie       attendu=valeur_non_numerique obtenu=valeur_non_numerique
OK durée nulle            attendu=valeur_non_positive obtenu=valeur_non_positive
OK distance négative      attendu=valeur_non_positive obtenu=valeur_non_positive
OK boucle                 attendu=boucle obtenu=boucle
OK abonnement inconnu     attendu=abonnement_inconnu obtenu=abonnement_inconnu
OK ordre : station avant valeur attendu=station_inconnue obtenu=station_inconnue

15/15 tests réussis

```

Figure : Tests unitaires des règles

Règle des doublons. Lorsqu'un même `trajet_id` apparaît plusieurs fois, seule la première occurrence dans l'ordre du fichier (numérotée avec `zipWithIndex`) est conservée.

Les deux lignes T000001 n'ont pas le même contenu : la ligne 2 est un trajet S08 → S19 le 2 janvier (40 min), la ligne 24006 un trajet S01 → S02 le 1er janvier à minuit (10 min). Ce n'est donc pas une copie qu'on pourrait supprimer sans réfléchir, mais un conflit : deux trajets différents avec le même identifiant. Il faut une règle pour choisir lequel garder.

Au moment de la lecture, `zipWithIndex` donne à chaque ligne son numéro de position dans le fichier. Ce numéro ne change jamais, quelle que soit la façon dont Spark découpe les données en partitions. En gardant toujours la plus petite position, on garde toujours la même ligne, à chaque exécution : la règle est déterministe.

Garder la date la plus ancienne aurait été une autre option, mais elle conserverait ici la ligne 24006 : une ligne de contrôle ajoutée en fin de fichier, datée pile de minuit le 1er janvier, donc suspecte.

L'énoncé précise que les cinq dernières lignes ne sont pas forcément les seules anomalies. Avant d'écrire les règles, le script `00_exploration.py` a donc contrôlé tout le fichier : nombre de colonnes (7 partout), champs vides (aucun), valeurs de `type_abonnement` (uniquement annuel, mensuel, occasionnel), plage de dates (du 01/01 au 31/03/2026) et vitesses moyennes (entre 7,2 et 16,8 km/h, réalistes). Aucune autre anomalie n'est apparue.

2.3 Bilan des rejets

Motif	Lignes	N° de ligne de données
format_invalide, date_invalide, hors_période, abonnement_inconnu	0	—
station_inconnue	1	24002 (S99)
valeur_non_numerique	1	24004 (durée = « inconnu »)
valeur_non_positive	1	24001 (durée = 0)
boucle	1	24003 (S03 → S03)
doublon_id	1	24005 (T000001, déjà vu en ligne de données 1)
Total rejeté	5	
Trajets valides	24 000	

Les numéros ci-dessus sont ceux affichés par le script (en-tête exclu) : la ligne de données n correspond à la ligne $n + 1$ du fichier CSV, celle qu'affiche `grep -n`.

```

=====
Stations chargées      : 36
Lignes de données     : 24005
Trajets valides       : 24000
Lignes rejetées       : 5
Contrôle valides+rejets : 24005 (= 24005 ?)
=====
BILAN DES REJETS PAR MOTIF (dans l'ordre d'application des règles)
format_invalide       0
date_invalide         0
hors_periode          0
station_inconnue      1
valeur_non_numerique  1
valeur_non_positive   1
boucle                1
abonnement_inconnu    0
doublon_id            1
=====
DÉTAIL DES LIGNES REJETÉES
ligne 24001 | valeur_non_positive | T024001,2026-03-12 08:10:00,S01,S02,0,1.2,annuel
ligne 24002 | station_inconnue   | T024002,2026-03-12 08:15:00,S01,S99,15,2.4,mensuel
ligne 24003 | boucle              | T024003,2026-03-12 08:20:00,S03,S03,12,1.9,occasionnel
ligne 24004 | valeur_non_numerique | T024004,2026-03-12 08:25:00,S05,S06,inconnu,2.1,annuel
ligne 24005 | doublon_id          | T000001,2026-01-01 00:00:00,S01,S02,10,1.5,annuel
=====

```

Figure : Bilan des rejets

2.4 Contrôle manuel

Quelques lignes du CSV ont été vérifiées à la main et comparées au résultat du pipeline :

Ligne	Vérification manuelle	Attendu	Pipeline
T000002	S26 et S04 existent, 36 min et 9,5 km positifs, S26 ≠ S04	valide	valide ✓
T000003	S17 et S14 existent, 64 min et 14,38 km positifs, S17 ≠ S14	valide	valide ✓
T000001 (ligne 2)	première occurrence de l'identifiant	gardée	gardée ✓
T000001 (ligne 24006)	identifiant déjà vu ligne 2	rejetée	doublon_id ✓

Le pipeline donne exactement le résultat attendu (capture 01).

```

[(.env)] abidhiahmed@MacBook-Pro-de-ABIDHIAF projet_trajets % cd ~/projet_trajets && grep -n "T000001" data/trajets.csv && sed -n '2,4p' data/trajets.csv
2:T000001,2026-01-02 06:29:00,S08,S19,40,5.03,mensuel
24006:T000001,2026-01-01 00:00:00,S01,S02,10,1.5,annuel
T000001,2026-01-02 06:29:00,S08,S19,40,5.03,mensuel
T000002,2026-02-02 18:25:00,S26,S04,36,9.5,mensuel
T000003,2026-02-21 12:49:00,S17,S14,64,14.38,occasionnel
[(.env)] abidhiahmed@MacBook-Pro-de-ABIDHIAF projet_trajets % █

```

Figure : Contrôle manuel

3. Indicateurs avec l'API RDD

Tous les indicateurs sont calculés sur les 24 000 trajets valides (script `02_rdd_indicateurs.py`), puis enrichis avec le nom des stations par `join`.

3.1 Départs et arrivées par station

Un `flatMap` transforme chaque trajet en deux événements, `((départ, "depart"), 1)` et `((arrivée, "arrivee"), 1)`, qui sont comptés par `reduceByKey` puis séparés par `filter`. Le résultat complet (36 stations) est dans `output/rdd_departs_arrivees.csv`.

	Min	Max
Départs	633 (S30)	724 (S14)
Arrivées	633 (S04, S08)	703 (S11, S19)

Contrôle de cohérence : total des départs = total des arrivées = 24 000, le nombre de trajets valides.

Contrôle manuel d'un indicateur : on recompte les départs de S01 directement dans le CSV, sans Spark. `awk` trouve 650 lignes avec `station_depart = S01`, dont 3 lignes rejetées (T024001, T024002 et le doublon T000001 de la dernière ligne). $650 - 3 = 647$, exactement la valeur calculée par le pipeline.

1) DÉPARTS ET ARRIVÉES PAR STATION				
station nom	zone	départs	arrivées	
S01	Station 01	1	647	694
S02	Station 02	1	686	688
S03	Station 03	1	670	649
S04	Station 04	1	682	633
S05	Station 05	1	670	654
S06	Station 06	1	647	656
S07	Station 07	1	672	697
S08	Station 08	1	647	633
S09	Station 09	1	649	655
S10	Station 10	1	636	700
S11	Station 11	1	644	703
S12	Station 12	1	652	688
S13	Station 13	2	668	671
S14	Station 14	2	724	665
S15	Station 15	2	679	680
S16	Station 16	2	711	662
S17	Station 17	2	655	659
S18	Station 18	2	679	672
S19	Station 19	2	652	703
S20	Station 20	2	654	638
S21	Station 21	2	689	635
S22	Station 22	2	665	670
S23	Station 23	2	667	642
S24	Station 24	2	648	690
S25	Station 25	3	716	693
S26	Station 26	3	675	683
S27	Station 27	3	674	663
S28	Station 28	3	657	661
S29	Station 29	3	659	671
S30	Station 30	3	633	648
S31	Station 31	3	673	645
S32	Station 32	3	654	645
S33	Station 33	3	640	675
S34	Station 34	3	674	658
S35	Station 35	3	662	674
S36	Station 36	3	690	647
Total départs = 24000,		total arrivées = 24000		

Figure : Départs/arrivées

3.2 Durée moyenne par zone de départ

Chaque trajet devient `(zone, (durée, 1))`. `reduceByKey` additionne les couples, et la division n'est faite qu'à la fin : une moyenne de moyennes partielles serait faussée si les partitions n'ont pas la même taille.

Zone	Somme (min)	Trajets	Moyenne (min)
1	315 584	7 902	39,94
2	318 854	8 091	39,41
3	317 285	8 007	39,63

Les trois zones ont des durées moyennes très proches, autour de 40 minutes.

3.3 Top 10 des couples départ–arrivée

Rang	Départ	Arrivée	Trajets
1	S27	S13	33
2	S08	S21	32
3	S35	S13	32
4	S05	S31	31
5	S07	S09	31
6	S09	S07	31
7	S15	S17	31
8	S16	S24	31
9	S16	S26	30
10	S32	S23	30

Quatre couples ont 30 trajets pour deux places restantes. Pour garder un classement déterministe, les égalités sont départagées par ordre alphabétique (départ puis arrivée). On remarque aussi que `S07 → S09` et `S09 → S07` sont tous les deux dans le top : c'est un axe utilisé dans les deux sens.

4. API DataFrame et couche Parquet

4.1 Schéma explicite et indicateurs

Le script `03_dataframe_parquet.py` lit les CSV avec un `StructType` explicite : les types (timestamp, double, string) sont déclarés au lieu d'être devinés par `inferSchema`. En mode `PERMISSIVE`, une valeur non convertible comme la durée « inconnu » devient `null`, puis est écartée par `dropna`. Les mêmes règles qu'en RDD sont réécrites avec des colonnes : `left_semi join` pour les stations connues, `filter` pour les valeurs positives et les boucles, et une fenêtre

`row_number()` par `trajet_id` pour garder la première occurrence. Résultat : **24 000 trajets valides**, comme en RDD. Les trois indicateurs de la partie 3 ont été recalculés avec `groupBy / agg / join`.

```

SCHEMA EXPLICITE DES TRAJETS
root
|-- trajet_id: string (nullable = true)
|-- date_heure: timestamp (nullable = true)
|-- station_depart: string (nullable = true)
|-- station_arrivee: string (nullable = true)
|-- duree_min: double (nullable = true)
|-- distance_km: double (nullable = true)
|-- type_abonnement: string (nullable = true)
|-- num_ligne: long (nullable = false)

Lignes lues : 24005 | trajets valides : 24000

DATAFRAME - DÉPARTS / ARRIVÉES (5 premières stations)
+-----+-----+-----+-----+
|station_id|nom|zone|nb_departs|nb_arrivees|
+-----+-----+-----+-----+
|S01|Station 01|1|647|694|
|S02|Station 02|1|686|688|
|S03|Station 03|1|670|649|
|S04|Station 04|1|682|633|
|S05|Station 05|1|670|654|
+-----+-----+-----+-----+
only showing top 5 rows

DATAFRAME - DURÉE MOYENNE PAR ZONE
+-----+-----+-----+-----+
|zone|somme_duree_min|nb_trajets|duree_moyenne_min|
+-----+-----+-----+-----+
|1|315584.0|7982|39.94|
|2|318854.0|8891|39.41|
|3|317285.0|8887|39.63|
+-----+-----+-----+-----+

```

Figure : Schéma DataFrame et premiers indicateurs

4.2 Écriture Parquet partitionnée par mois

Une colonne `mois (yyyy-MM)` est ajoutée, puis les trajets nettoyés sont écrits avec `partitionBy("mois")` dans `output/parquet/trajets/`. Spark crée un sous-dossier par mois ; une requête filtrée sur un mois ne lit que le dossier concerné (*partition pruning*).

Partition	Trajets
mois=2026-01	8 286
mois=2026-02	7 375
mois=2026-03	8 339
Total	24 000

La relecture du Parquet redonne bien 24 000 trajets.

4.3 Comparaison RDD / DataFrame

Le script compare automatiquement les sorties des deux versions : départs/arrivées, durée moyenne par zone et top 10 sont **identiques** (True pour les trois).

Critère	RDD	DataFrame
Schéma	implicite (namedtuple Python, types vérifiés à la main)	explicite (StructType), vérifié à la lecture
Conversion / erreurs	try/except dans une fonction Python	null en mode PERMISSIVE, puis dropna
Lisibilité	tuples imbriqués (<code>kv[1][0][2]</code>), plus difficile à relire	colonnes nommées, proche du SQL
Optimisation	aucune : Spark exécute exactement le code écrit	optimiseur Catalyst (plan logique → physique)
Contrôle	total, bas niveau	moins fin mais suffisant ici
Résultats	référence	identiques ✓

En pratique, la version DataFrame est plus courte et plus lisible : les colonnes ont un nom, le schéma est contrôlé dès la lecture, et Catalyst optimise le plan sans intervention. La version RDD reste utile pour comprendre ce que Spark fait réellement (chaque `map`, chaque shuffle est écrit explicitement) et pour les traitements difficiles à exprimer en colonnes, comme la validation ligne par ligne avec un motif de rejet précis. Dans ce projet, les deux API donnent exactement les mêmes résultats.

5. Plan d'exécution et optimisation

5.1 Transformations étroites et larges

Une transformation **étroite** calcule chaque partition de sortie à partir d'une seule partition d'entrée : pas d'échange réseau.

Une transformation **large** a besoin des données de plusieurs partitions : Spark doit faire un *shuffle* et démarre un nouveau stage.

Transformation (où dans le pipeline)	Type	Shuffle ?
map (parsing et validation, commun.py)	étroite	non
filter (séparation valides / rejets)	étroite	non
flatMap (événements départ/arrivée, étape 2)	étroite	non
mapValues (couples (durée, 1))	étroite	non
reduceByKey (comptages, sommes)	large	oui
join (doublons, enrichissement avec les noms)	large	oui
groupByKey (benchmark)	large	oui
groupBy().agg() (DataFrame)	large	oui (Exchange)

5.2 Lecture du plan et de l'interface Spark

Lignée RDD (toDebugString, capture 07). Chaque +- marque une frontière de stage, c'est-à-dire un shuffle. Pour le simple comptage des départs, on voit trois shuffles : le reduceByKey de la détection des doublons, le join qui garde la première occurrence et le reduceByKey final. Les lignes CachedPartitions montrent que le RDD des trajets valides est lu depuis la mémoire (667 Kio) au lieu d'être recalculé.

```

1) LIGNÉE RDD (toDebugString) - départs par station
(4) PythonRDD[29] at RDD at PythonRDD.scala:53 []
| MapPartitionsRDD[28] at mapPartitions at PythonRDD.scala:160 []
| ShuffledRDD[27] at partitionBy at NativeMethodAccessorImpl.java:0 []
+- (4) PairwiseRDD[26] at reduceByKey at /Users/abidhiahmed/projet_trajets/src/04_plan_benchmark.py:46 []
| PythonRDD[25] at reduceByKey at /Users/abidhiahmed/projet_trajets/src/04_plan_benchmark.py:46 []
| PythonRDD[23] at RDD at PythonRDD.scala:53 []
| CachedPartitions: 4; MemorySize: 667.0 KiB; DiskSize: 0.0 B
| MapPartitionsRDD[19] at mapPartitions at PythonRDD.scala:160 []
| ShuffledRDD[18] at partitionBy at NativeMethodAccessorImpl.java:0 []
+- (4) PairwiseRDD[17] at join at /Users/abidhiahmed/projet_trajets/src/commun.py:82 []
| PythonRDD[16] at join at /Users/abidhiahmed/projet_trajets/src/commun.py:82 []
| UnionRDD[15] at union at NativeMethodAccessorImpl.java:0 []
| PythonRDD[13] at RDD at PythonRDD.scala:53 []
| PythonRDD[8] at RDD at PythonRDD.scala:53 []
| CachedPartitions: 2; MemorySize: 1405.2 KiB; DiskSize: 0.0 B
| data/trajets.csv MapPartitionsRDD[5] at textFile at NativeMethodAccessorImpl.java:0 []
| data/trajets.csv HadoopRDD[4] at textFile at NativeMethodAccessorImpl.java:0 []
| PythonRDD[14] at RDD at PythonRDD.scala:53 []
| MapPartitionsRDD[12] at mapPartitions at PythonRDD.scala:160 []
| ShuffledRDD[11] at partitionBy at NativeMethodAccessorImpl.java:0 []
+- (2) PairwiseRDD[10] at reduceByKey at /Users/abidhiahmed/projet_trajets/src/commun.py:81 []
| PythonRDD[9] at reduceByKey at /Users/abidhiahmed/projet_trajets/src/commun.py:81 []
| PythonRDD[8] at RDD at PythonRDD.scala:53 []
| CachedPartitions: 2; MemorySize: 1405.2 KiB; DiskSize: 0.0 B
| data/trajets.csv MapPartitionsRDD[5] at textFile at NativeMethodAccessorImpl.java:0 []
| data/trajets.csv HadoopRDD[4] at textFile at NativeMethodAccessorImpl.java:0 []
=====

```

Figure : Lignée RDD

Plan physique DataFrame (explain, capture 07b). Pour la durée moyenne par zone, Catalyst produit :

- un Scan parquet qui ne lit que les 2 colonnes utiles (ReadSchema: struct<station_départ, durée_min>) : c'est l'avantage du format colonne ;
- un BroadcastHashJoin : la petite table des 36 stations est envoyée à chaque tâche (BroadcastExchange), ce qui évite un shuffle de la grande table ;
- un HashAggregate partiel (partial_avg = somme + nombre par partition), puis un seul Exchange hashpartitioning(zone, 200), puis l'agrégat final. C'est exactement le principe (somme, nombre) de la partie 3.2, appliqué automatiquement.

```

2) PLAN PHYSIQUE DATAFRAME - durée moyenne par zone
== Physical Plan ==
AdaptiveSparkPlan (13)
+- HashAggregate (12)
  +- Exchange (11)
    +- HashAggregate (10)
      +- Project (9)
        +- BroadcastHashJoin Inner BuildRight (8)
          :- Project (3)
          : +- Filter (2)
          : +- Scan parquet (1)
        +- BroadcastExchange (7)
          +- Project (6)
            +- Filter (5)
            +- Scan csv (4)

```

Figure : Plan physique

Interface Spark (capture 08). L'onglet *Stages* montre les colonnes *Shuffle Write* et *Shuffle Read*. Par exemple, le

comptage des départs par station n'écrit que 2,1 Kio de shuffle, alors que le comptage des couples départ-arrivée en écrit 45,1 Kio : il y a beaucoup plus de clés distinctes (1 260 couples distincts contre 36 stations).

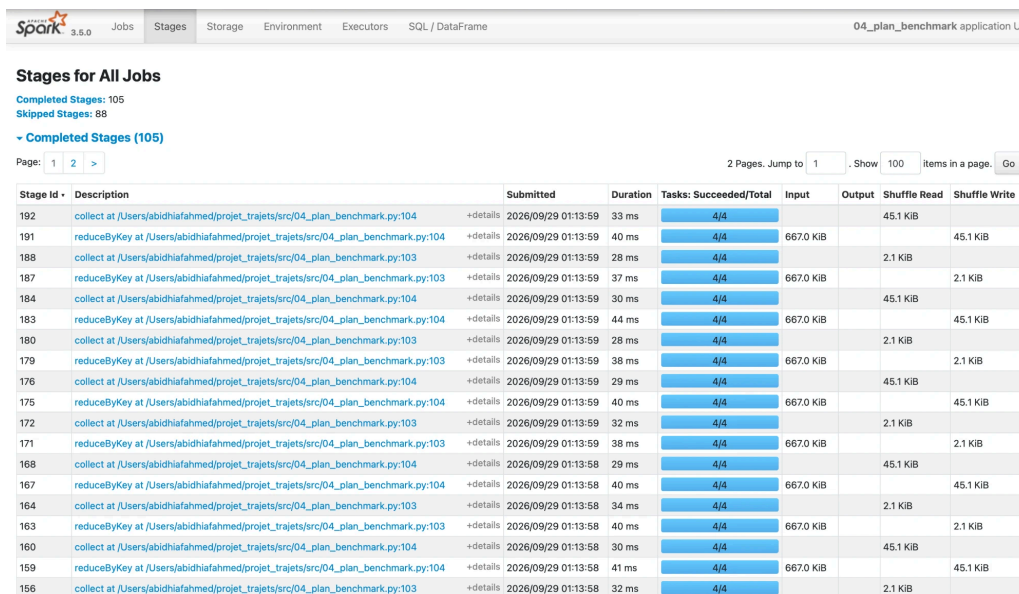


Figure : Spark UI - Stages

5.3 groupByKey vs reduceByKey, partitionnement et cache

Les mesures ont été faites sur le même Mac, en local [*], avec 1 exécution d'échauffement puis 5 exécutions chronométrées par variante (output/benchmark.csv).

Variante	Run 1	Run 2	Run 3	Run 4	Run 5	Médiane (s)
RDD groupByKey	0,094	0,096	0,089	0,086	0,091	0,091
RDD reduceByKey	0,085	0,089	0,085	0,084	0,089	0,085
DataFrame – 200 partitions de shuffle	0,018	0,015	0,016	0,016	0,015	0,016
DataFrame – 8 partitions de shuffle	0,017	0,016	0,016	0,016	0,017	0,016
Deux indicateurs sans cache	0,205	0,210	0,208	0,199	0,199	0,205
Deux indicateurs avec cache	0,170	0,175	0,167	0,170	0,169	0,170

Volume écrit par le shuffle, relevé via l'API REST de l'interface Spark (output/shuffle_volumes.csv) :

Variante	Shuffle write
groupByKey	74 963 octets (73,2 Kio)
reduceByKey	3 087 octets (3,0 Kio)

groupByKey vs reduceByKey. Les temps sont très proches (6 ms d'écart sur la médiane), mais le volume échangé est environ **24 fois plus faible** avec `reduceByKey`. En effet, `groupByKey` envoie les 24 000 durées sur le réseau avant de les additionner, alors que `reduceByKey` combine d'abord localement dans chaque partition et n'envoie qu'un couple (somme, nombre) par station et par partition. Sur 24 000 lignes et une seule machine, ce gain ne se voit presque pas en temps ; sur un vrai cluster, le shuffle passe par le réseau et le disque, et c'est lui qui coûte le plus cher.

Partitionnement. Par défaut, un shuffle DataFrame crée 200 partitions, beaucoup trop pour 3 zones. Avec 8 partitions, le temps ne change pas ici, car l'*Adaptive Query Execution* (`AdaptiveSparkPlan` dans le plan) fusionne automatiquement les partitions vides après le shuffle. Côté RDD, le fichier est lu en 2 partitions puis 4 après les jointures : ce nombre fixe le parallélisme maximal (une tâche par partition, visible dans la colonne *Tasks 4/4*).

Cache. Sans `cache()`, chaque action relit le CSV et refait toute la validation (parsing, doublons, jointure). Avec le cache, les deux indicateurs repartent du RDD en mémoire : médiane de 0,205 s à 0,170 s, soit environ 17 % de moins. Le cache est pertinent ici parce que le même RDD nettoyé est réutilisé par plusieurs indicateurs.

Limite. Ces mesures portent sur un petit jeu de données, sur une seule machine, et les écarts sont de quelques millisecondes. Elles ne prouvent pas une accélération générale : elles illustrent seulement des mécanismes (volume de shuffle, recalcul évité) qui deviennent importants à plus grande échelle.

6. Graphe orienté du réseau

Le graphe est construit en RDD (05_graphe.py) : les **sommets** sont les 36 stations, et chaque **arc** départ → arrivée

porte comme poids le nombre de trajets, obtenu par `reduceByKey` sur les couples `((départ, arrivée), 1)`. Résultat : 36 sommets, **1 260 arcs**, pour un poids total de 24 000 trajets.

Constat important. $1\,260 = 36 \times 35$: le graphe est **complet**. Chaque station envoie et reçoit des trajets de toutes les autres, donc tous les degrés non pondérés valent 35 (entrant et sortant). Le degré simple ne permet pas de distinguer les stations.

Définition annoncée. Une station est d'autant plus connectée que son **degré total pondéré** est élevé, c'est-à-dire le nombre de trajets entrants + sortants. En cas d'égalité, on départage par le degré non pondéré, puis par l'identifiant.

Rang	Station	Zone	Entrants	Sortants	Degré total pondéré
1	S25	3	693	716	1 409
2	S14	2	665	724	1 389
3	S02	1	688	686	1 374
4	S16	2	662	711	1 373
5	S07	1	697	672	1 369

Ces valeurs sont cohérentes avec la partie 3.1 : pour S25, 716 départs + 693 arrivées = 1 409. Les trois zones sont représentées dans le top 5, et les écarts restent faibles (1 369 à 1 409, soit moins de 3 %) : le réseau synthétique est très homogène. Les arcs les plus lourds sont les mêmes que le top 10 de la partie 3.3 (S27 → S13 : 33 trajets). Les degrés complets et la liste des arcs sont dans `output/graphe_degres.csv` et `output/graphe_arcs.csv`.

```
Toutes les stations ont deg_in = deg_out = 35 : True

TOP 5 DES STATIONS LES PLUS CONNECTÉES (degré total pondéré = trajets entrants + sortants)
1. S25 (Station 25) : 1409 trajets (entrants 693, sortants 716)
2. S14 (Station 14) : 1389 trajets (entrants 665, sortants 724)
3. S02 (Station 02) : 1374 trajets (entrants 688, sortants 686)
4. S16 (Station 16) : 1373 trajets (entrants 662, sortants 711)
5. S07 (Station 07) : 1369 trajets (entrants 697, sortants 672)

ARCS LES PLUS LOURDS (5 premiers)
S27 -> S13 : 33
S08 -> S21 : 32
S35 -> S13 : 32
S05 -> S31 : 31
S07 -> S09 : 31
```

Figure : Top 5 graphe

7. Limites et conclusion

Le pipeline va des CSV bruts jusqu'aux indicateurs et au graphe, de façon reproductible : 5 lignes sont rejetées avec un motif explicite, 24 000 trajets sont validés, et les versions RDD et DataFrame donnent des résultats identiques. Les données nettoyées sont stockées en Parquet partitionné par mois.

Limites :

- **Données synthétiques et homogènes.** Le graphe complet et les durées moyennes quasi identiques par zone montrent un réseau sans vraie structure. Sur des données réelles, les différences entre stations seraient plus marquées.
- **Mesures de performance.** Elles sont faites sur un seul Mac en mode local, avec un petit volume : elles illustrent des mécanismes (volume de shuffle, cache) sans prouver un gain général.
- **Règle des doublons.** « Première occurrence » est déterministe mais repose sur l'ordre du fichier. En RDD, `zipWithIndex` le garantit ; en DataFrame, `monotonically_increasing_id` le respecte ici parce qu'il n'y a qu'un seul fichier source. Si les données arrivaient de plusieurs fichiers, il faudrait une autre clé d'ordre, par exemple une date d'ingestion.
- **Pas de table transactionnelle.** Le Parquet ne gère ni mises à jour ni historique. Une couche Delta Lake ou Iceberg serait l'étape suivante pour un vrai lakehouse.